

Lab 7: Multi-Region Topologies & Survival Goals (65 min)

Learning Objectives

By the end of this lab you will be able to:

- Spin up a simulated 3-region, 9-node cluster in Docker with `demo --global`, including inter-region latency
- Toggle survival between `ZONE FAILURE` and `REGION FAILURE` and measure write latency
- Convert tables to `REGIONAL BY TABLE`, `REGIONAL BY ROW`, and `GLOBAL` and observe per-region read latency
- Take a region offline and confirm the cluster keeps serving
- Use `crdb_internal.zones` and `SHOW RANGES WITH DETAILS` to audit replica placement
- Choose the right locality for a given access pattern via a decision tree

Prerequisites

- **Docker Desktop** (or Docker Engine) running — there is no `cockroach` binary to install
- Enough RAM to run 9 in-memory nodes (~6 GB)
- Lab 5 / 6 not required, but Lab 1's concepts assumed

Setup

```
docker run --rm -it -p 8090:8080 \  
  cockroachdb/cockroach:v23.2.5 \  
  demo --global --nodes 9 --no-example-database --empty --http-port=8080
```

That drops you straight into a SQL shell inside the container. The DB Console is at <http://localhost:8090>.

The `\demo ...` commands below only work in this interactive shell. They are client-side commands, not SQL: pipe them into a non-interactive session and you get `ERROR: invalid syntax: \demo connect 2`. Keep this shell open and type them here. To reach a specific node from a *script* instead, connect to that node's own port — `demo` assigns them sequentially from 26257, so node 4 is 26260.

Why demo here, and not the compose cluster? This is the one lab that needs *simulated inter-region latency* — `--global` inserts realistic round-trip delays between its nine simulated regions, which is the whole point of measuring a cross-region write. Nine real containers on one laptop would all be a sub-millisecond hop apart and the latency lesson would vanish. `demo` also ships a temporary licence, which multi-region SQL requires.

It is self-contained: nothing to install, and it disappears on exit. Stop the lab cluster first if you want the RAM back (`scripts/crdb down`).

⚠ **Multi-region SQL is an enterprise feature.** `SET PRIMARY REGION` , `ADD REGION` , `SURVIVE ... FAILURE` , `REGIONAL BY ROW` , and `LOCALITY GLOBAL` all require a licence, and without one you get use of multi-region features requires an enterprise license . `cockroach demo` includes a **temporary licence**, which is why this lab uses it. If you run these steps against your own `cockroach start` cluster instead, set a trial licence first:

```
SET CLUSTER SETTING enterprise.license = 'crl-0-...';
```

(No `cluster.organization` — the class key is bound to an empty organization name, and setting one invalidates it.) Node **localities** (`--locality=region=...`) and zone configurations are core features and work on any cluster.

You should see something like:

node	region	zone
1	us-east1	us-east1-a
2	us-east1	us-east1-b
3	us-east1	us-east1-c
4	us-west1	us-west1-a
5	us-west1	us-west1-b
6	us-west1	us-west1-c
7	europe-west1	europe-west1-a
8	europe-west1	europe-west1-b
9	europe-west1	europe-west1-c

Confirm:

```
SELECT node_id, locality FROM crdb_internal.gossip_nodes ORDER BY node_id;
```

The `--global` flag also injects simulated inter-region latency (~80–100ms RTT) so the locality benefits are observable.

Tasks

Part A: Pick a Survival Goal, Measure the Cost (15 min)

1. Create a regional database and load the regions:

```
CREATE DATABASE shop;  
USE shop;  
  
ALTER DATABASE shop SET PRIMARY REGION "us-east1";  
ALTER DATABASE shop ADD REGION "us-west1";  
ALTER DATABASE shop ADD REGION "eu-west1";  
  
SHOW REGIONS FROM DATABASE shop;
```

2. Default survival is **ZONE FAILURE** :

```
SHOW SURVIVAL GOAL FROM DATABASE shop;
```

3. Create a simple table and time some inserts:

```
CREATE TABLE products (  
  id      UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  name    STRING NOT NULL,  
  price   DECIMAL(10,2) NOT NULL  
);  
  
\set show_times  
INSERT INTO products (name, price)  
SELECT 'item-' || g, (random()*100)::DECIMAL(10,2)  
FROM generate_series(1, 100) g;
```

Replicas are within **us-east1**. Writes have low intra-region latency.

4. Switch to **REGION FAILURE** :

```
ALTER DATABASE shop SURVIVE REGION FAILURE;
```

Replication factor goes from 3 → 5, spread across all three regions.

5. Re-run the inserts:

```
INSERT INTO products (name, price)  
SELECT 'item2-' || g, (random()*100)::DECIMAL(10,2)  
FROM generate_series(1, 100) g;
```

Each insert is slower — cross-region RTT shows up in the commit path.

6. Switch back to **ZONE FAILURE** to keep the rest of the lab snappy:

```
ALTER DATABASE shop SURVIVE ZONE FAILURE;
```

Part B: REGIONAL BY ROW (15 min)

1. Create a table with row-level locality:

```
CREATE TABLE customers (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  email       STRING UNIQUE,  
  name        STRING NOT NULL,  
  region      crdb_internal_region NOT NULL  
) LOCALITY REGIONAL BY ROW AS region;
```

2. Insert customers in each region:

```
INSERT INTO customers (email, name, region) VALUES  
('alice@us.example.com', 'Alice', 'us-east1'),  
('bob@us.example.com', 'Bob', 'us-west1'),  
('charlie@eu.example.com', 'Charlie', 'europe-west1'),  
('dana@us.example.com', 'Dana', 'us-east1');
```

3. Verify each row's range leaseholder is in its region:

```
SELECT email, region FROM customers ORDER BY email;  
SHOW RANGES FROM TABLE customers WITH DETAILS;
```

Compare each range's `lease_holder_locality` to the row's region — they should match.

4. Connect to a `us-east1` node and time a regional read:

```
\demo connect 1
```

```
\set show_times  
SELECT name FROM customers WHERE email = 'alice@us.example.com'; -- local  
SELECT name FROM customers WHERE email = 'charlie@eu.example.com'; -- remote
```

5. Connect to a European node and watch the pattern flip:

```
\demo connect 7
```

```
SELECT name FROM customers WHERE email = 'alice@us.example.com'; -- now remote  
SELECT name FROM customers WHERE email = 'charlie@eu.example.com'; -- now local
```

6. Give a related table the same home. Tables never share a range, so the only cross-table locality available is the region — and only if each table carries the same region value. Create `orders` `REGIONAL BY ROW`, copying the customer's region onto each order:

```
CREATE TABLE orders (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  customer_id UUID NOT NULL REFERENCES customers(id),
  total       DECIMAL(12,2) NOT NULL,
  region      crdb_internal_region NOT NULL
) LOCALITY REGIONAL BY ROW AS region;

INSERT INTO orders (customer_id, total, region)
SELECT id, 42.00, region FROM customers;
```

First the *request* — one partition per region, each with a lease preference for it:

```
SELECT partition_name, zone_config FROM [SHOW PARTITIONS FROM TABLE orders];
```

Then the *result* — where the allocator actually put the leaseholders:

```
SELECT 'customers' AS tbl, start_key, lease_holder_locality
FROM [SHOW RANGES FROM TABLE customers WITH DETAILS]
UNION ALL
SELECT 'orders', start_key, lease_holder_locality
FROM [SHOW RANGES FROM TABLE orders WITH DETAILS]
ORDER BY 2, 1;
```

Partitions appear in region order (the key encodes the region enum). Give the allocator about two minutes after creating the tables (measured) and re-run: each partition's leaseholder locality converges to its own region, for both tables alike. A join for one customer now stays inside that region. Without the copied column, `orders` would default to the *gateway's* region — and a European customer who ordered while travelling would carry a cross-region join forever.

Part C: GLOBAL Tables for Reference Data (10 min)

For lookup tables read everywhere but rarely written.

1. Reconnect to node 1:

```
\demo connect 1
```

2. Create a GLOBAL table:

```
CREATE TABLE country_codes (
  code  STRING(2) PRIMARY KEY,
  name  STRING NOT NULL
) LOCALITY GLOBAL;

INSERT INTO country_codes (code, name) VALUES
('US', 'United States'),
('CA', 'Canada'),
('DE', 'Germany'),
('FR', 'France'),
('GB', 'United Kingdom');
```

3. Time the same read from three regions:

```
\demo connect 1
```

```
\set show_times  
SELECT name FROM country_codes WHERE code = 'US';
```

```
\demo connect 4
```

```
SELECT name FROM country_codes WHERE code = 'US';
```

```
\demo connect 7
```

```
SELECT name FROM country_codes WHERE code = 'US';
```

All three reads should be fast — GLOBAL tables read locally everywhere.

4. But writes are slow:

```
\set show_times  
UPDATE country_codes SET name = 'United States of America' WHERE code = 'US';
```

GLOBAL writes have to wait for clocks to advance past a future timestamp — slower than REGIONAL writes.

Part D: REGIONAL BY TABLE — Pinned to One Region (10 min)

Sometimes you want a small reference table to live entirely in one region (think: legal-residency requirements).

1. Create a REGIONAL BY TABLE pinned to Europe:

```
CREATE TABLE eu_pricing_rules (  
  id    UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  rule  STRING NOT NULL  
) LOCALITY REGIONAL BY TABLE IN "europe-west1";  
  
INSERT INTO eu_pricing_rules (rule) VALUES  
  ('VAT 19%'), ('VAT 20%'), ('VAT 21%');
```

2. Compare access from regions:

```
\demo connect 7
```

```
\set show_times  
SELECT * FROM eu_pricing_rules;           -- local in EU (fast)
```

```
\demo connect 1
```

```
SELECT * FROM eu_pricing_rules;           -- remote (slow)
```

Part E: Take a Region Offline (10 min)

Now the proof. With `SURVIVE REGION FAILURE` set, does the cluster keep serving when a region disappears?

1. Set the database to `SURVIVE REGION FAILURE` so it can actually:

```
\demo connect 1
```

```
ALTER DATABASE shop SURVIVE REGION FAILURE;
```

Wait ~30 s for re-replication.

2. List europe nodes:

```
SELECT node_id FROM crdb_internal.gossip_nodes  
WHERE locality LIKE '%europe-west1%';
```

3. Take all three EU nodes offline:

```
\demo shutdown 7  
\demo shutdown 8  
\demo shutdown 9
```

4. Verify the cluster is still serving (reconnect to a survivor if needed):

```
\demo connect 1
```

```
SELECT count(*) FROM customers;  
INSERT INTO customers (email, name, region)  
VALUES ('eve@us.example.com', 'Eve', 'us-east1');  
SELECT email, region FROM customers ORDER BY email;
```

5. What about querying a European customer's row?

```
SELECT email, region FROM customers WHERE email = 'charlie@eu.example.com';
```

With `SURVIVE REGION FAILURE`, the leaseholder has been promoted to a surviving region; reads succeed. Give it ~10s after the outage to fully transition.

6. Bring Europe back:

```
\demo restart 7
\demo restart 8
\demo restart 9
```

In the DB Console's **Replication** dashboard, watch under-replicated ranges drop back to 0.

Part F: Audit Replica Placement (5 min)

1. For each **REGIONAL BY ROW** row, where do its replicas live?

```
SELECT
  range_id,
  start_key,
  lease_holder,
  replicas,
  lease_holder_locality,
  replica_localities
FROM [SHOW RANGES FROM TABLE customers WITH DETAILS]
ORDER BY start_key;
```

2. **Zone configs in the system catalog** — this is the mechanism under everything you did in Parts A–D:

```
SHOW ZONE CONFIGURATION FROM TABLE customers;
SHOW ZONE CONFIGURATIONS;
```

REGIONAL BY ROW did not place anything itself. It wrote one zone configuration per region partition, with **constraints** and **lease_preferences** expressed in locality terms (**+region=europe-west1**), and the **allocator** satisfies them by matching against what each node advertised in **--locality**. Put the two side by side:

```
SELECT node_id, locality FROM crdb_internal.gossip_nodes ORDER BY node_id;
-- what nodes offer
```

The **region=** strings in the second must match the **+region=** strings in the first exactly.

The failure is asymmetric. **ADD REGION** for a name no node advertises is refused immediately. A node started with a *mis-typed* locality — **region=us-east-1** where the database says **us-east1** — is not refused: it joins, reports healthy, and simply never receives the replicas its region's zone configs ask for. The symptom is a region whose ranges are all led from elsewhere. These two queries are how you find it.

3. For the database overall:

```
SELECT * FROM crdb_internal.zones WHERE target = 'DATABASE shop';
```

Optional — If Time Allows

These parts are not required to complete the lab; they extend it by about 10 minutes. Do them if you finish early, or after class — the cluster and data from the core parts carry over.

Part G: Decision Tree (10 min)

For each row, pick the locality and survival goal.

Workload	Locality	Survival
App-config table read from any region; rarely written	?	?
Per-tenant SaaS data; tenant has a home region	?	?
Audit log written by every region; never read	?	?
Compliance-pinned EU customer data	?	?
Reference tax rates updated weekly, read everywhere	?	?
Shopping-cart row that follows the user across regions (rare)	?	?
Reporting dashboard over orders; five seconds stale is acceptable	?	?
Global leaderboard written from every region and read as current	?	?

Defend each choice. Then build them out on this cluster and confirm via `SHOW RANGES`. Two of these rows have no "local everywhere" answer — say which, and what you would pin instead.

Cleanup

```
DROP DATABASE shop CASCADE;
```

`\q` exits the SQL shell; the cluster keeps running.

The cluster keeps running between labs — that is the point of it being persistent. To wipe everything and start fresh at any time:

```
scripts/crdb reset
```

Lab 7 Deliverables

☐ **Multi-region cluster** with 9 nodes across 3 regions ☐ **Survival goals tested**: measured the latency tradeoff ZONE vs REGION ☐ **Locality applied**: REGIONAL BY ROW, REGIONAL BY TABLE IN, GLOBAL verified via per-region reads ☐ **Region outage survived**: 3 of 9 nodes (one entire region) offline; cluster kept

serving ☐ **Placement audited**: read replica localities from `SHOW RANGES` and `crdb_internal.zones` ☐
Decision tree applied to six concrete scenarios

Challenge Exercises

1. **Design a ride-share dataset.** Trips happen in regions and should be read/written locally. Vehicles (~5k entries) are looked up from anywhere. Pricing rules (~50 entries) are updated weekly. What locality + survival do you pick? Build it; verify with `SHOW RANGES`.
2. **Measure GLOBAL write cost.** Time 100 sequential updates to your `country_codes` table. Compare against 100 inserts into a REGIONAL BY TABLE. The gap is your cost-of-globalness.
3. ****A "wrong" choice.** What happens if you set `LOCALITY GLOBAL` on a high-write table? Try with `orders` analogous to Lab 4. Predict, then verify, the throughput hit.

Reference

Locality	Best for	Read latency	Write latency
<code>REGIONAL BY TABLE IN x</code>	Region-pinned reference data	Fast in x, slow elsewhere	Fast in x
<code>REGIONAL BY ROW</code>	Per-row home region	Fast in that row's region	Fast in that row's region
<code>GLOBAL</code>	Read-anywhere, rarely-written	Fast everywhere	Cross-region (slow)

Survival	Replicas	Tolerates
<code>ZONE FAILURE</code> (default)	3	Loss of 1 zone in primary region
<code>REGION FAILURE</code>	5	Loss of 1 entire region